深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

Big Data Analytics
大数据分析 [2026 国际商务]

Lecture 6 — Logistic Regression and Linear Classifier 第六讲·逻辑回归与线性分类器
从最小二乘到极大似然，再到梯度下降与正则化

任课教师：戴谢尔 (Xieer Dai, David)
深圳大学 Shenzhen University

·第 3–14 周·星期四 11-12 节
上课地点：四方楼南 209



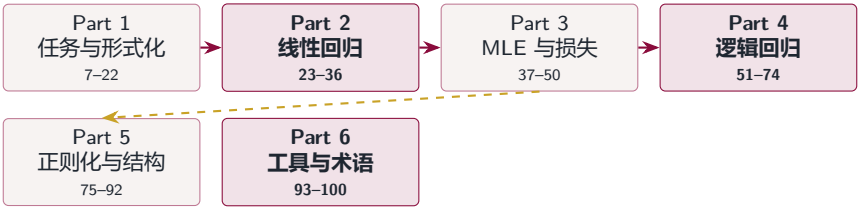
学习目标 Learning Objectives

完成本讲后，你应能够：

1. 写出线性回归的正规方程 normal equation 并推导它；
2. 用极大似然估计 MLE 导出 l_2 损失与交叉熵 cross-entropy；
3. 解释 sigmoid 的三条性质（有界、对称、$\sigma' = \sigma(1-\sigma)$）；
4. 手写梯度下降 gradient descent 并与 PyTorch 自动微分 对照；
5. 解释对数几率 log-odds 与几率比 odds ratio 的业务含义；
6. 用正则化 regularisation（LASSO/Ridge/Elastic Net）与概率校准 calibration 控制复杂度与可信度。

来源：对应课程大纲第 6 讲目标：掌握线性回归的闭式解与极大似然推导；掌握逻辑回归、交叉熵与梯度下降。

本讲路线图 Roadmap（100 帧）



主线：任务 → 形式化 → 最小二乘 → 极大似然 → 分类 → 正则化

代码页 10 帧•样本页 14 帧：术语全程中英对照，代码逐行英文注释

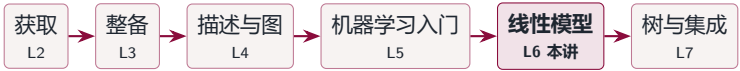
与上一讲的衔接 From Evaluation to Estimation

L5 结束时你手上有什么

- 学习范式、偏差-方差、评估协议与算法族地图;
- 但还有一个问题没答：损失函数为什么是平方、是交叉熵?

L6 要回答的问题

- 函数类怎么选（假设类 hypothesis class)?
- 损失从哪来（似然 likelihood)?
- 参数怎么解（正规方程或梯度下降)?
- 怎么控制复杂度（正则化)?



本讲是推导课：四条推导（正规方程、l_2 = 高斯 + MLE、逻辑回归 = sigmoid + MLE、交叉熵梯度）都会给完整步骤。

本讲输出物 Deliverables

1) 四条推导

正规方程；l_2 = 高斯 + MLE；逻辑回归 = sigmoid + MLE；$\nabla L = \frac{1}{n} X^\top (\sigma(Xw) - y)$

2) 手写实现

NumPy 正规方程与梯度下降，逐行英文注释；与 scikit-learn、statsmodels、PyTorch 对账

3) 术语表

79 条中英对照（CSV + Word/Excel）；每条含符号、公式与一句话解释

本讲的数字都来自可复现的程序：`website/code/lecture06/`，种子固定 20260603，一条命令重建全部图与表。

来源：程序与样本随课程仓库发布；术语表另附可打印版本。

全讲导览 Map of the Lecture

部分	帧	关键问题
任务与形式化	7–22	什么是学习任务？假设类、损失与经验风险怎么定义？
线性回归与正规方程	23–36	最小二乘怎么解？为什么是投影？数值稳不稳？
极大似然与损失函数	37–50	平方损失与交叉熵分别对应什么概率假设？
线性分类与逻辑回归	51–74	为什么用 sigmoid？怎么解？怎么解释系数？
正则化与监督学习结构	75–92	LASSO 与 Ridge 差在哪？偏差-方差怎么分解？
工具、术语与结语	93–100	推断视角怎么用？术语怎么背？

读法：每部分是" 定义 → 推导 → 样本图 → 可运行代码 → 术语" 的循环；术语首次出现都附英文。

三类任务与核心术语 Three Tasks and Their Terms

二分类

二分类 binary classification
输出两个类别，例如欺诈/正常、室内/室外。
训练数据 training data、测试数据 test data

多分类

多分类 multi-class classification
输出三个及以上类别，例如 ImageNet 的上千类。

聚类

聚类 clustering
无标签数据分组；无监督学习 unsupervised learning

一个例子把三个术语串起来（图像分类）

任务：判断图片是室内 indoor 还是室外 outdoor；性能度量：误分类概率 probability of misclassification；经验/数据：带标签的图片（有标签数据 labeled data）。

术语复习：监督学习 supervised learning 用带标签数据；无监督学习 unsupervised learning 只用特征。

来源：术语与例子沿用课程讲义的任务设定；英文对照据机器学习通行译法。

图像分类：任务与性能度量 Image Classification

任务

给定一张图片，判断它是室内还是室外。这是一个二分类 binary classification 问题。

性能度量

最自然的选择是误分类概率 probability of misclassification：$\Pr[f(x) \neq y]$。它正是0-1 损失的期望。

为什么不用准确率

准确率 $= 1-$ 误分类率；两者等价。但误判的代价未必对称：把" 室外" 误判为" 室内" 与反过来，业务后果可能差很多 (衔接 Part 4 的第一类错误 Type I error 与第二类错误 Type II error)。

本节要建立一个习惯：先写清任务与度量，再选模型。度量一旦写错，后面所有推导都会跑偏。

来源：例子据课程讲义；代价不对称的说明衔接 L5 的评估协议。

多分类与真实规模 Multi-class at Scale

从二分类到多分类

多分类 multi-class classification：输出 $K > 2$ 个类别。做法有两种常见路线：

- 训练 K 个"一对多"二分类器（one-vs-rest）；
- 直接输出 K 个概率（softmax），即多分类逻辑回归。

真实规模

ImageNet 有上千个类别、上千万张图片。此时：

- 类别越多，混淆矩阵越大，单指标越不够用；
- 需要top-k 准确率与每类召回率；
- 训练与推理的成本成为设计约束。

预告：softmax 与多类交叉熵在帧 68–70 展开，本帧只建立"多分类是分类的自然延伸"这一直觉。

来源：ImageNet 规模为公开事实；多分类评估的提醒为本讲补充。

聚类：无标签数据的分组 Clustering

任务与度量

任务：把图片分成两组；性能：组内相似度similarity高、组间差异大。数据：一组无标签unlabeled 图片。

两组术语

- 无标签数据 unlabeled data vs 有标签数据 labeled data；
- 无监督学习 unsupervised learning vs 监督学习 supervised learning。

与 L5 的呼应：无监督没有" 正确答案"，所以评估只能靠内部指标与业务解释；本讲不展开聚类算法，只强调术语与评估的差别（详见 L8）。

来源：例子据课程讲义；评估提醒衔接 L5 的无监督部分。

从像素到特征向量 From Pixels to a Feature Vector

手工特征

用颜色直方图 colour histogram 概括一张图片：把红/绿/蓝三个通道各分若干区间，统计落在每个区间的像素比例，得到一串数字——这就是特征 feature。

数学记号

- 特征向量 feature vector $x_i \in \mathbb{R}^d$；
- 标签 label y_i（分类里 $y_i \in \{0, 1\}$）；
- 训练集写作 $\{(x_i, y_i) : 1 \le i \le n\}$。

" 提取特征" 这一步决定了模型的上限。本讲先假设特征已给定，到帧 21 再讨论" 能不能让机器自己学特征"（表征学习 representation learning）。

来源：颜色直方图的例子据课程讲义；记号为本讲统一约定。

形式化（1）：找一个函数 Find a Function

问题

给定训练数据 training data $\{(x_i, y_i) : 1 \le i \le n\}$，找一个函数 f，使得 $y = f(x)$ 对新数据也成立。

为什么说"新数据"

在训练集上拟合得好没有意义——把每个样本都记住即可。真正要的是泛化 generalisation：在未见样本上正确。

这一步还缺什么

- 候选函数从哪里来？（下一步：假设类 hypothesis class）
- "正确"怎么衡量？（后面的损失函数 loss function）

提示：形式化一共六步（帧 12–18），每一步只加一个要素，最后拼成完整的学习问题。

来源：六步形式化据课程讲义的组织顺序；说明为本讲补充。

形式化（2）：假设类 Hypothesis Class

加入假设类

在给定的函数族 $\mathcal{H}$ 中找 $f \in \mathcal{H}$。$\mathcal{H}$ 称为假设类 hypothesis class，它决定了模型的表达能力。

三个例子

- 线性函数：$\mathcal{H} = \{x \mapsto w^\top x\}$——最常用、可解释；
- 多项式：$\mathcal{H} = \{x \mapsto \sum_{j \le d} w_j x^j\}$——更灵活但更易过拟合；
- 神经网络：$\mathcal{H}$ 由层数与激活函数决定。

术语要点

选 $\mathcal{H}$ 就是选模型族 model family；本讲后面所有推导都限定在线性 $\mathcal{H}$ 上，便于得到闭式解。

来源：假设类的定义统计学习通行表述；三个例子为本讲补充。

形式化（3）：训练与测试同分布 Same Distribution

加入分布假设

训练数据与测试数据都来自同一个分布 D。没有这个假设，" 在训练集上学到的规律" 就不保证对新数据有效。

这个假设意味着什么

- 训练集与测试集是同一个世界的两个样本；
- 数据的生成机制在两者之间没有变化；
- 一旦环境变化（政策、疫情、平台规则），假设就破了。

破了会怎样

分布漂移 distribution shift：样本外误差突然变大。这正是 L5 的 MLOps 监控要盯的东西（衔接帧 96—97）。

来源：同分布假设为统计学习的基本前提；漂移的后果衔接 L5 的监控与评测部分。

形式化（4）：独立同分布 i.i.d.

加入独立性

训练样本 $\{(x_i, y_i)\}$ 与测试样本都独立同分布（i.i.d. independently identically distributed）：同分布给出" 同一个世界"，独立给出" 样本之间不互相决定"。

独立性为什么重要

有了独立性，经验平均才是期望的无偏近似，大数定律与集中不等式才能用来给出泛化界。

什么时候不成立

时间序列（今天的价格依赖昨天）、面板数据（同一公司多年）、社交网络（用户互相影响）——本讲末的股市例子（帧 22）就是反例。

来源：i.i.d. 的作用据统计学习教材；不成立的场景为本讲补充（衔接帧 22）。

形式化（5）：期望损失 Expected Loss

加入性能度量

要求期望损失（期望风险 expected risk）小：

$$L(f) = \mathbb{E}_{x,y\sim D}\big[l(f,x,y)\big],$$

其中 l 是损失函数 loss function。它衡量" 平均而言错得多离谱"。

两种常用损失

- 0-1 损失 0-1 loss：$l = \mathbb{I}[f(x) \neq y]$，取期望即误分类概率；
- 平方损失 l_2 loss：$l = (f(x) - y)^2$，取期望即均方误差。

注意

期望损失是对分布求的，我们并不知道 D，所以只能估计——这就引出下一步的经验损失 empirical loss。

来源：期望风险的定义给出学习通行表述。

损失函数示例：0-1 与 l_2 Two Loss Functions

损失	单样本形式	期望形式与用途
0-1 损失 0-1 loss	$\mathbb{I}[f(x) \neq y]$	$L(f) = \Pr[f(x) \neq y]$；分类、可直接解释
平方损失 l_2 loss	$(f(x) - y)^2$	$L(f) = \mathbb{E}[(f(x) - y)^2]$；回归、易求导
l_1 损失 l_1 loss	$\lvert f(x) - y \rvert$	对离群点更稳健；不可导点需处理
合页损失 hinge loss	$\max(0, 1 - yf(x))$	支持向量机的损失（L7）
指数损失 exponential loss	$e^{-yf(x)}$	AdaBoost 的损失（L7）

一个问题先记下：为什么线性回归偏偏用 l_2？帧 31 给经验理由，帧 43 给概率理由（MLE）。

来源：损失家族据统计学习教材；本讲聚焦 0-1 与 l_2。

形式化（6）：经验损失 Empirical Loss

用样本平均代替期望

既然不知道 D，就用训练集上的平均——经验损失 empirical loss（经验风险 empirical risk）：

$$\hat{L}(f) = \frac{1}{n} \sum_{i=1}^{n} l(f, x_i, y_i).$$

整个学习问题

给定 $\{(x_i, y_i)\}$ i.i.d. $\sim D$，在假设类 $\mathcal{H}$ 中找 f 使 $\hat{L}(f)$ 最小。这就是经验风险最小化 empirical risk minimisation (ERM)。

一句话记住

学习 = 在 $\mathcal{H}$ 里找 f，让 $\hat{L}(f)$ 最小。

来源：ERM 的表述统计学习通行定义。

期望风险与经验风险 The Generalisation Gap

两个风险

- $L(f)$：期望风险 expected risk，我们真正关心的；
- $\hat{L}(f)$：经验风险 empirical risk，我们能算的；
- 二者之差 $L - \hat{L}$ 叫泛化差距 generalisation gap。

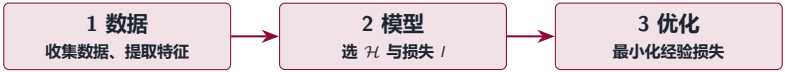
两种失败

- 差距大 $\rightarrow$ 过拟合 overfitting：模型记住了训练噪声；
- 两者都大 $\rightarrow$ 欠拟合 underfitting：假设类太简单。

这就是 L5 的偏差-方差故事在这里的" 风险版本"：训练好 $\neq$ 泛化好，所以必须留出验证集（衔接 L5 帧 26–30）。

来源：泛化差距与两种失败据统计学习教材；与 L5 的衔接为本讲补充。

机器学习 1-2-3 Machine Learning in Three Steps



三个要素与术语

- 数据：特征提取 feature extraction、经验 experience；
- 模型：假设类 hypothesis class + 损失函数 loss function；
- 优化：经验风险最小化 ERM，常见手段是梯度下降 gradient descent。

先验知识在哪

先验知识 prior knowledge 会进入前两步：选什么特征、选什么假设类，都是人做的判断——这也是为什么 L5 说"AI 不替代判断"。

来源：1-2-3 框架沿用课程 L1/L5 的表述。

能不能让机器自己学特征 Representation Learning

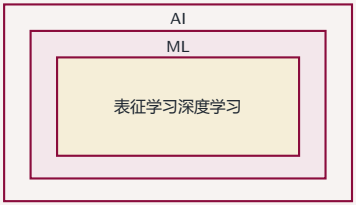
疑问

为什么还要手工设计特征向量 x？能不能直接在原始图片上学习特征？

答案

可以，这称为表征学习 representation learning：让模型自己学习从原始输入到有用表示的映射。

包含关系



DL $\subseteq$ Representation Learning $\subseteq$ ML $\subseteq$ AI

2026 的说法：预训练模型就是大规模表征学习（衔接 L9 深度学习与 L11 自然语言处理）。

来源：包含关系据课程讲义；2026 视角为本讲补充。

1-2-3 覆盖不了什么 What the Recipe Misses

并非所有方法都符合 1-2-3

- 强化学习 reinforcement learning：没有固定的训练集，数据由交互产生；
- 主动学习 active learning：自己决定标注哪些样本；
- 在线学习 online learning：数据流式到达，模型持续更新。

反例：股市预测

用滑动窗口 sliding window 构造样本：用过去 k 天预测下一天。但相邻样本高度相关、分布随时间漂移，i.i.d. 不成立，所以" 随机切分" 会给出虚高的样本外表现（衔接 L5 的泄漏清单）。

结论：1-2-3 是" 默认模板"，用之前先问三件事——数据独立吗？同分布吗？有标签吗？

来源：边界与反例据课程讲义与 L5 的数据泄漏讨论。

一个真实数据集：前列腺癌 Prostate Cancer Data

数据

Stamey et al. (1989) 的 97 位病人的临床数据：

- 目标 y：前列腺特异性抗原 prostate specific antigen (PSA) 的对数；
- 特征 $(x_1, \ldots, x_8)$：临床指标 clinical measures ——对数癌体积、对数前列腺重量、年龄、对数 BPH、精囊侵犯、对数包膜穿透、Gleason 评分、Gleason 4/5 百分比。

为什么用它

这是《The Elements of Statistical Learning》的经典例子：样本不大（$n = 97$）、特征不多（$p = 8$）、关系近似线性——正好用来演示最小二乘、推断与正则化的全部细节。

本讲的样本 `16_prostate.csv` 按同样的变量结构合成（数值非真实病人数据），以便复现且不涉及隐私；图注中会注明这一点。

来源：数据来源：Stamey, T. et al. (1989), *Journal of Urology*; 经 ESL 教材广泛使用。

线性回归：假设类与损失 Linear Regression

形式化

给定训练数据 $\{(x_i, y_i) : 1 \le i \le n\}$ i.i.d. $\sim D$，在线性假设类中找

$$f_w(x) = w^\top x, \qquad w \in \mathbb{R}^d,$$

并使平方损失 l_2 loss 的经验风险 $\hat{L}(f_w) = \frac{1}{n}\sum_i (w^\top x_i - y_i)^2$ 最小。

术语

- 线性模型 linear model：f_w 关于 w 线性；
- 均方误差 mean squared error (MSE)：即 $\hat{L}$ 本身；
- 残差 residual：$y_i - w^\top x_i$。

注意

"f 关于 x 线性" 是假设，不是定理；真实关系弯曲时，线性模型会欠拟合 (L5 帧 32)。

来源：形式化与记号据统计学习教材；术语为本讲统一。

矩阵形式：把求和写成向量运算 Matrix Form

构造

把 n 个样本堆成矩阵：第 i 行为 $x_i^\top$，即

$$X \in \mathbb{R}^{n\times d}, \qquad y \in \mathbb{R}^n.$$

于是

$$\hat{L}(f_w) = \frac{1}{n}\|Xw - y\|_2^2$$

$$= \frac{1}{n}\left(w^\top X^\top X w - 2w^\top X^\top y + y^\top y\right).$$

展开时用到的两个恒等式：$(Xw)^\top(Xw) = w^\top X^\top X w$，以及 $(Xw)^\top y = w^\top X^\top y$。

为什么这样写

- 求和变成矩阵乘法，便于求导；
- 复杂度从 $O(nd)$ 的循环变成一次矩阵运算；
- 后续的梯度、正规方程都能直接用矩阵表达。

来源：矩阵形式据统计学习教材的标准推导。

梯度为零：正规方程的推导 The Normal Equation

推导

对 w 求梯度并令其为零：

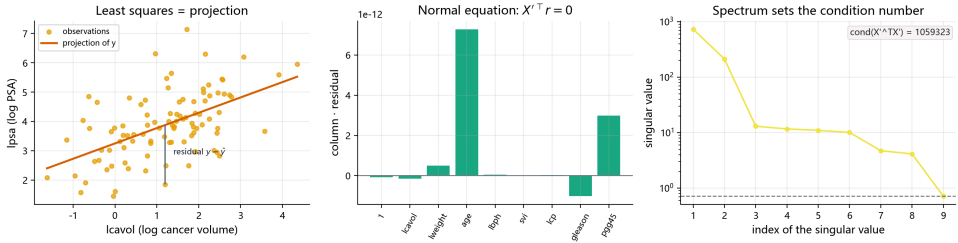
$$
\begin{aligned}
\nabla_w \hat{L} &= \nabla_w \frac{1}{n}\left(w^\top X^\top X w - 2w^\top X^\top y + y^\top y\right) \\
&= \frac{1}{n}\left(2X^\top X w - 2X^\top y\right) = 0 \\
\Longrightarrow \quad & X^\top X w = X^\top y \\
\Longrightarrow \quad & \boxed{w = (X^\top X)^{-1} X^\top y}
\end{aligned}
$$

这组方程叫 正规方程 normal equation；解 w 叫最小二乘解 least squares solution。条件是 $X^\top X$ 可逆（列满秩）。

Set the gradient to zero, solve the linear system.

来源：推导据统计学习教材；本讲把每一步都写出来便于跟算。

【样本 + 可视化】正规方程的几何：投影 Projection



左：最小二乘解是 y 在 X 的**列空间**上的投影 projection，残差与列空间正交；中：数值验证 $X^\top r \approx 0$（本样本最大点积 7×10^{-12}）；右：设计矩阵的**奇异值谱**，其比值决定条件数 condition number。

程序 `16_01_linreg_normal_equation.py` ·样本 `16_prostate.csv` ·系数表 `16_coef_compare.csv`

带偏置项：增广技巧 The Bias Term

模型

加上截距 intercept（也叫偏置 bias）：

$$f_{w,b}(x) = w^\top x + b.$$

化归为无偏置情形

令

$$w' = \begin{pmatrix} w \\ b \end{pmatrix}, \qquad x' = \begin{pmatrix} x \\ 1 \end{pmatrix},$$

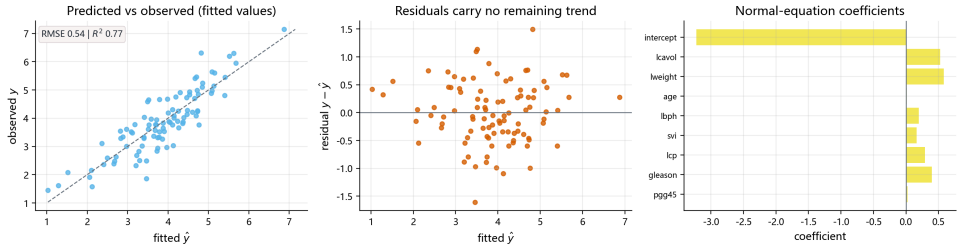
则 $f_{w,b}(x) = w'^\top x'$——形式完全一样，只是多了一列常数。

实现上的三点

- 增广矩阵记作 $X' = [\mathbf{1}, X]$；
- 这一列不要标准化（否则常数项被压成 0）；
- 正则化时通常不惩罚截距。

来源：增广技巧据统计学习教材；实现细节为工程实践补充。

【样本 + 可视化】前列腺数据：拟合、残差与系数



本讲合成样本（$n = 97$，$p = 8$）：RMSE 0.54、R^2 0.77；残差对拟合值没有明显趋势；右图给出正规方程解出的八个系数（对数尺度上，`lcavol` 与 `lweight` 的影响最大）。

程序 `16_01_linreg_normal_equation.py` ·三种实现（NumPy / sklearn / statsmodels）系数完全一致

【代码】正规方程的三种实现 Three Implementations

```
Xa = add_intercept(X)                                  # augmented design matrix X' = [1, X]
w_np = normal_equation(Xa, y)                          # w = (X'^T X')^{-1} X'^T y by our helper
w_sk = LinearRegression(fit_intercept=False).fit(Xa, y).coef_ # scikit-learn on the augmented matrix
w_sm = sm.OLS(y, Xa).fit().params                      # statsmodels ordinary least squares
max_gap = float(np.max(np.abs(w_np - w_sk)))           # largest gap between two implementations
assert max_gap < 1e-8, "implementations disagree"      # they must agree to numerical precision
y_hat = Xa @ w_np                                      # fitted values
resid = y - y_hat                                      # residuals
orthogonality = Xa.T @ resid                           # dot products of residuals with the columns
print("max |column . residual| = %.2e" % float(np.max(np.abs(orthogonality)))) # should be ~0
coef_table = pd.DataFrame({"term": ["intercept"] + cols, # term names in the augmented order
                           "numpy": np.round(w_np, 4),    # normal-equation solution
                           "sklearn": np.round(w_sk, 4),  # scikit-learn solution
                           "statsmodels": np.round(w_sm, 4)}) # statsmodels solution
dump(coef_table, "l6_coef_compare.csv")                # persist the comparison table
```

要点：三种实现必须给同一组系数（本样本差小于 10^{-8}）；残差与设计矩阵正交是正规方程的几何签名。完整程序 `l6_01_linreg_normal_equation.py`。

为什么偏偏是 l_2 损失 Why the Squared Loss

经验理由：好优化

平方损失对 w 是二次函数：梯度是线性的，令其为零得到闭式解 $w = (X^\top X)^{-1} X^\top y$。换 l_1 或合页损失就没有这个便利。

理论理由：编码先验

平方损失等价于假设噪声服从高斯分布（帧 43 会证明）。所以" 用 l_2" 其实是" 假设残差是高斯白噪声"——这是一个可检验的建模假设。

术语：先验知识 prior knowledge 在这里的具体含义是" 误差的分布形状"。选择损失就是选择你对数据的假设，而不是选一个" 更好看的函数"。

来源：两条理由据统计学习教材与课程讲义；MLE 的证明见帧 43。

多重共线性与条件数 Multicollinearity

问题

当两列近似线性相关时，$X^\top X$ 接近奇异：

- 条件数 condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ 很大；
- 系数对数据的微小扰动极其敏感；
- 系数符号可能与本意相反（" 符号翻转"）。

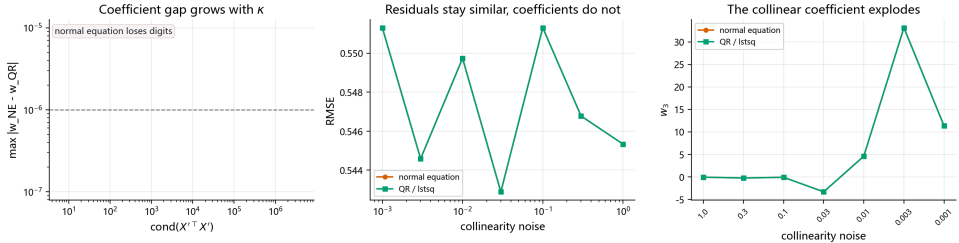
三条对策

1. 用QR 分解或 SVD 求解，而不是显式求逆（帧 34）；
2. 标准化特征，改善数值尺度；
3. 用正则化（Ridge）接受一点偏差换取稳定（帧 78）。

本讲实测：前列腺样本的 $\kappa(X'^\top X') \approx 1.06 \times 10^6$——已经很高，说明先标准化再解释系数是必要的。

来源：条件数与共线性据统计学习教材；实测数值来自程序 l6_01。

【样本 + 可视化】数值稳定性：正态方程 vs QR



让第三列越来越接近第一列：$\kappa(X'^{\top}X')$ 上升，正态方程与 QR 的系数差按数量级放大；但两者的残差几乎相同——说明" 拟合得好" 不能证明" 系数算得对"。

程序 `16_02_numerical_stability.py` ·样本 `16_illconditioned.csv` ·扫描表 `16_stability.csv`

【代码】数值稳定：lstsq、QR 与伪逆 Stable Least Squares

```
gram = design.T @ design                              # gram matrix X'ᵀX'
cond = float(np.linalg.cond(gram))                    # condition number of the normal equations
w_ne = normal_equation(design, y)                     # normal-equation solution
w_qr = np.linalg.lstsq(design, y, rcond=None)[0]      # QR/SVD based solution
w_sv = np.linalg.pinv(design) @ y                     # explicit pseudo-inverse solution
resid_ne = rmse(y, design @ w_ne)                     # residual of the normal-equation fit
resid_qr = rmse(y, design @ w_qr)                     # residual of the stable fit
rows.append({"collinearity_noise": level, "cond_gram": cond, # record the condition number
             "coef_gap_ne_vs_qr": float(np.max(np.abs(w_ne - w_qr))), # largest coefficient gap
             "rmse_ne": resid_ne, "rmse_qr": resid_qr, # residuals of both solutions
             "w_x3_ne": float(w_ne[3]), "w_x3_qr": float(w_qr[3])}) # the unstable coefficient itself
stability = pd.DataFrame(rows).round(6)               # assemble the table
dump(stability, "l6_stability.csv")                   # persist it
```

工程结论：永远用 `lstsq/solve`，不要显式写 `inv(X'X)`；伪逆 pseudo-inverse 还能处理秩亏（X 不满列秩）的情形。完整程序 `l6_02_numerical_stability.py`。

线性回归的评估 Evaluation and Extrapolation Risk

三个诊断

- R^2 coefficient of determination：解释了多少方差；
- 残差图 residual plot：看是否还有结构（弯曲 = 缺项）；
- RMSE root mean squared error：与业务单位一致的误差。

外推的代价

线性模型在训练数据的取值范围内插值较可靠；超出范围（外推 extrapolation）时，$\hat{y}$ 会线性地越走越远，可能与现实完全不符。

实践建议：报告里写清训练数据的取值区间；对超出区间的预测要标注" 外推，仅作参考"。

来源：诊断与风险据统计学习实践；外推提醒为本讲补充。

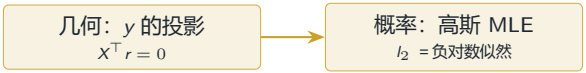
小结：最小二乘的两种解释 Two Readings of Least Squares

几何：投影

$\hat{y} = Xw$ 是 y 在 X 列空间上的正交投影；残差 $r = y - \hat{y}$ 与列空间正交，因此 $X^\top r = 0$——这正是正规方程。

概率：极大似然

若 $y \mid x \sim \mathcal{N}(w^\top x, \sigma^2)$，则最小化平方损失 $\Leftrightarrow$ 最大化似然（帧 43 证明）。



下一部分把"概率"这条线展开：损失的来源就是似然。

来源：两种解释据统计学习教材；与 Part 3 的衔接为本讲安排。

为什么需要极大似然 Why Maximum Likelihood

问题

损失函数不是随手挑的。它编码了你对数据如何产生的假设。如果损失与数据生成过程不匹配，模型就会系统性地错。

MLE 的思路

极大似然估计 maximum likelihood estimation (MLE)：在所有候选参数中，选那个让已观测数据出现概率最大的参数。

这一部分的目标：把" 损失函数" 从" 给定" 变成" 推导出来"。推导完你会看到——平方损失是高斯假设的产物，交叉熵是伯努利假设的产物。

来源：MLE 的动机据统计学习教材；框架为本讲组织。

分布族与参数 Distributions and Parameters

设定

给定训练数据 $\{(x_i, y_i)\}$，假设数据来自一族由参数 θ 索引的分布：

$$\{P_\theta(x, y) : \theta \in \Theta\}.$$

θ 是参数 parameter，Θ 是参数空间 parameter space。

例子

- 高斯族：$\theta = (\mu, \sigma^2)$；
- 伯努利族：$\theta = p$；
- 逻辑回归：$\theta = w$，$P_w(y = 1 \mid x) = \sigma(w^\top x)$。

关键区分

参数是要学的 (w)；超参数 (hyper-parameter) 是人定的（如 λ、学习率）。

来源：分布族的表述统计学习通行定义；超参数的区分衔接帖 76。

似然：从单个样本到整份数据 Likelihood

单个样本的"拟合度"

定义参数 θ 对一个样本 (x_i, y_i) 的似然likelihood：

$$\mathrm{lik}(\theta; x_i, y_i) := P_\theta(x_i, y_i).$$

它是"在参数为 θ 时，看到这个样本的概率"。

整份数据（利用 i.i.d.）

独立同分布意味着概率相乘：

$$\mathrm{lik}(\theta; \{(x_i, y_i)\}) = \prod_{i=1}^{n} P_\theta(x_i, y_i).$$

这就是联合似然joint likelihood。

注意术语差别：概率是"给定参数看数据"，似然是"给定数据看参数"——同一个式子，两种读法。

来源：似然的定义统计学习通行表述。

为什么取对数 Why Take the Log

两个理由

1. 数值：n 个小于 1 的数相乘会下溢 underflow 成 0（帧 48）；
2. 代数：对数把乘积变成求和，求导方便。

$$\log \mathrm{lik}(\theta) = \sum_{i=1}^{n} \log P_\theta(x_i, y_i).$$

单调性保证等价

$\log$ 是单调递增函数，所以

$$\arg\max_\theta \mathrm{lik}(\theta) = \arg\max_\theta \log \mathrm{lik}(\theta).$$

最大化似然与最大化对数似然 log-likelihood 是同一件事。

来源：取对数的理由据统计学习与数值计算实践；单调性为基本事实。

负对数似然作为损失 Negative Log-Likelihood

把”最大化”改成”最小化”

优化器通常做最小化，于是定义负对数似然 negative log-likelihood (NLL)：

$$\hat{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log P_\theta(x_i, y_i).$$

所以

极大似然估计 $\Leftrightarrow$ 最小化 NLL。而 NLL 正是我们要找的损失函数——它由分布假设唯一决定（差一个常数因子）。

两个具体的 NLL

- 高斯 $\rightarrow$ 平方损失（帧 43）；
- 伯努利 $\rightarrow$ 逻辑损失，即交叉熵 cross-entropy（帧 44、59）。

来源：ML 与 ERM 的关系统计学习通行表述。

条件似然：判别式与生成式 Conditional Likelihood

判别式 discriminative

只建模条件分布：

$$P_\theta(y \mid x).$$

目标是最小化 $-\frac{1}{n}\sum_i \log P_\theta(y_i \mid x_i)$。分类任务通常这么做（逻辑回归）。

生成式 generative

建模联合分布：

$$P_\theta(x, y) = P_\theta(x) P_\theta(y \mid x).$$

除了分类，还能生成样本、处理缺失；代价是假设更多、参数更多。

逻辑回归是判别式的：它只关心" 给定特征下标签的概率"，不假设特征本身怎么分布。这使它对特征分布的错误假设更稳健。

来源：判别式与生成式的区分据 Bishop 与统计学习教材；与本讲的关系为本讲补充。

【推导】平方损失 = 高斯噪声 + MLE l_2 Loss from a Gaussian

推导（三步）

设 $y_i \mid x_i \sim \mathcal{N}(w^\top x_i, \sigma^2)$，则

$$P_w(y_i \mid x_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(- \frac{(y_i - w^\top x_i)^2}{2\sigma^2} \Big).$$

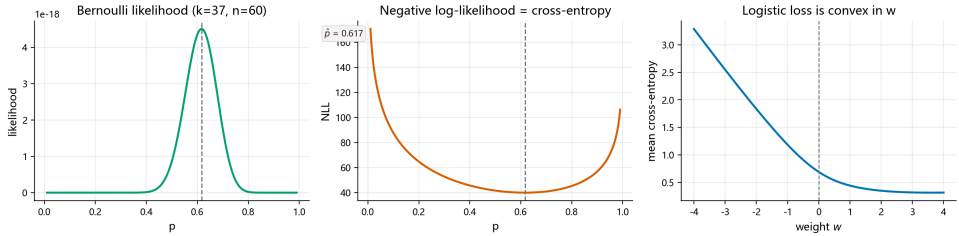
取负对数，与 w 无关的项是常数；对 i 求和、除以 n、乘 $2\sigma^2$ 后只剩平方项：

$$\arg\min_w \big[- \log \mathrm{lik} \big] = \arg\min_w \frac{1}{n} \sum_i (y_i - w^\top x_i)^2.$$

结论：在高斯噪声假设下，最小化平方损失就是极大似然。所以" 为什么用 l_2" 的答案不是审美，而是对误差分布的假设。

来源：推导统计数据学习教材；步骤为本讲展开。

【样本 + 可视化】伯努利似然与逻辑损失 Bernoulli Likelihood



左：伯努利似然在观测频率处取最大（本样本 $k = 37$、$n = 60$，MLE $\hat{p} = 0.617$）；中：对应的负对数似然 NLL 是凸的；右：单个特征的逻辑损失 logistic loss 也是凸函数——凸性保证梯度下降收敛到全局最优。

程序 `16_03_mle_gaussian.py` ·结果 `16_spam.csv` ·与帧 59 的交叉熵推导衔接

交叉熵与 KL 散度 Cross-Entropy and KL Divergence

定义

对两个分布 p, q：

$$H(p,q) = -\sum_k p_k \log q_k \quad (\text{交叉熵 cross-entropy}),$$

$$D_{KL}(p\|q) = \sum_k p_k \log \frac{p_k}{q_k}.$$

两式满足 $H(p,q) = H(p) + D_{KL}(p\|q)$。

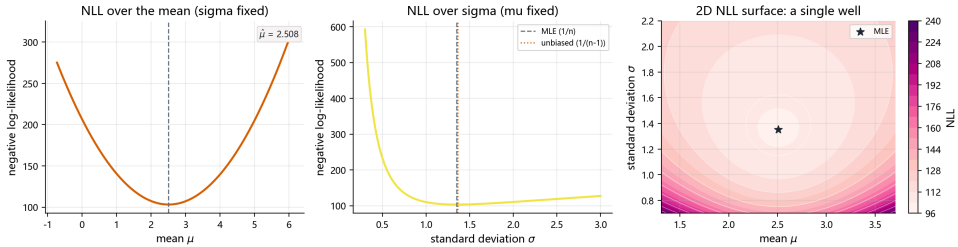
为什么分类用它

当 p 是真实标签 (one-hot) 时，$H(p)$ 是常数，于是最小化交叉熵 $\Leftrightarrow$ 最小化 KL 散度 $\Leftrightarrow$ 让预测分布逼近真实分布。

二分类时交叉熵写成 $-\frac{1}{n}\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]$，其中 $p_i = \sigma(w^\top x_i)$——这就是帧 59 要推导的逻辑回归损失。

来源：交叉熵与 KL 的关系据信息论与机器学习教材。

【样本 + 可视化】负对数似然曲面 The NLL Surface



高斯样本（$n = 60$，真实 $\mu = 2.5, \sigma = 1.4$）：左：固定 σ 时 NLL 对 μ 是抛物线，最优解是样本均值 2.302；中：NLL 对 σ 的最优点在 1.367（除以 n），而无偏估计是 1.379（除以 $n-1$）；右：二维曲面是单个" 碗"，凸且唯一最优点。

程序 `16_03_mle_gaussian.py` ·曲面数据 `16_nll_surface.csv` ·与数值优化的结果一致

【代码】用 MLE 拟合高斯 Fitting a Gaussian by MLE

```python
def nll_gaussian(params, data):                          # negative log-likelihood of a Gaussian
    mu, log_sigma = params                               # unpack the parameter vector
    sigma = np.exp(log_sigma)                            # keep sigma positive by construction
    z = (data - mu) / sigma                              # standardised residuals
    return float(np.sum(0.5 * z ** 2 + log_sigma + 0.5 * np.log(2 * np.pi))) # sum of NLL terms
fit = minimize(nll_gaussian, x0=[0.0, 0.0], args=(sample,), method="Nelder-Mead") # numerical MLE
mu_grid = np.linspace(sample.min() - 0.5, sample.max() + 0.5, 300) # grid over the mean
nll_mu = [nll_gaussian([m, np.log(mle_sd)], sample) for m in mu_grid] # NLL as a function of the mean
sd_grid = np.linspace(0.3, 3.0, 300)                     # grid over the standard deviation
nll_sd = [nll_gaussian([mle_mu, np.log(s)], sample) for s in sd_grid] # NLL as a function of sigma
mle_mu = float(sample.mean())                            # MLE of the mean equals the sample mean
mle_sd = float(sample.std(ddof=0))                       # MLE of the sd divides by n, not n-1
print("numerical MLE: mu %.4f | sigma %.4f" % (fit.x[0], np.exp(fit.x[1]))) # should match the closed form
```

要点：参数化要对——用 $\log\sigma$ 保证 $\sigma > 0$；闭式解与数值优化必须一致，这是检验推导的最快方式。完整程序 `l6_03_mle_gaussian.py`。

数值问题：下溢与 log-sum-exp Underflow

下溢

300 个 0.01 相乘是 10^{-600}，在浮点数里直接变成 0；似然变成 0 后，对数无定义、梯度全为 0。

溢出

反过来，$\exp(1000)$ 会变成 inf，softmax 里出现 inf/inf 就得到 NaN。

解法：先减最大值

$$\log \sum_k e^{z_k} = m + \log \sum_k e^{z_k - m}, \quad m = \max_k z_k.$$

同理 softmax 用 $e^{z_k - m}$ 计算。这个技巧叫 log-sum-exp log-sum-exp。

本讲实测：300 个 0.01 的连乘得到 0.0，而取对数再求和得到 -1381.55；logits 为 $\{1000, 1001, 1002\}$ 时，朴素 softmax 返回 nan，稳定版返回 $\{0.09, 0.24, 0.67\}$。

来源：下溢与溢出为浮点数基本事实；数值来自程序 l6_04。

【代码】稳定的 log-sum-exp Stable Log-Sum-Exp

```python
def stable_softmax(z):                          # shifted softmax used in practice
    z = np.asarray(z, dtype=float)              # work with a float array
    shifted = z - np.max(z)                     # the shift removes the overflow
    ez = np.exp(shifted)                        # safe exponentials
    return ez / np.sum(ez)                      # normalise
def logsumexp(a, axis=None, keepdims=False):    # stable log-sum-exp helper
    a = np.asarray(a, dtype=float)              # work with float arrays
    amax = np.max(a, axis=axis, keepdims=True)  # subtract the maximum first
    shifted = np.exp(a - amax)                  # safe exponentials
    out = amax + np.log(np.sum(shifted, axis=axis, keepdims=True)) # log-sum-exp of the shifted values
    return float(np.squeeze(out)) if axis is None and not keepdims else out # scalar when fully reduced
naive_product = float(np.prod(probs))           # naive product underflows to zero
log_product = float(np.sum(np.log(probs)))      # working in logs keeps the value
assert naive_product == 0.0, "expected underflow for 300 factors of 0.01" # document the failure mode
```

要点：任何概率连乘、任何 softmax，都应先减最大值；这条规则在深度学习的损失函数里无处不在（衔接 L9）。完整程序 `l6_04_logsumexp.py`。

小结：损失函数从哪里来 Where Losses Come From

损失	对应的分布假设	推导出的解
平方损失 l_2	高斯噪声 $\mathcal{N}(w^\top x, \sigma^2)$	闭式解 $w = (X^\top X)^{-1} X^\top y$
交叉熵（二分类）	伯努利 $P(y \mid x) = \sigma(w^\top x)$	无闭式解，用梯度下降
交叉熵（多分类）	类别分布 + softmax	无闭式解，用梯度下降
l_1 **损失**	拉普拉斯噪声	中位数回归，无闭式解

一句话：损失 = 你对数据的概率假设。下一部分把" 伯努利 + sigmoid" 这条线走完，得到逻辑回归与它的梯度。

来源：损失与分布的对应关系统计学习与信息论教材。

线性分类的几何 Geometry of a Linear Classifier

决策规则

用超平面把空间分成两半：

$$\hat{y} = \begin{cases} 1, & w^\top x > 0 \\ 0, & w^\top x < 0 \end{cases}$$

w 是法向量 normal vector，$w^\top x = 0$ 是超平面 hyperplane（决策边界）。

两个几何量

- 间隔 margin：样本到边界的距离 $|w^\top x|/\|w\|$；
- 间隔越大，分类越"自信"——这也是 L7 支持向量机的出发点。

从回归到分类只换了两件事：输出从实数变成类别，损失从平方变成误分类计数。后者带来一个新困难（帧 53）。

来源：线性分类的几何据统计学习教材；间隔的铺垫衔接 L7。

自然尝试：阶跃函数 + 0-1 损失 The Natural Attempt

写法

用阶跃函数 step function 把线性得分变成类别：

$$\hat{y} = \mathrm{step}(w^\top x), \qquad \hat{L}(w) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}\big[\mathrm{step}(w^\top x_i) \neq y_i\big].$$

这就是 0-1 损失 0-1 loss 的经验风险。

垃圾邮件例子（输入形态）

每封邮件变成一个向量：叹号数、“Mr.” 数、“sale” 数、…
标签是 spam / not spam。

直觉上很合理

误分类计数最贴合任务目标：错一个算一个。问题在于——它没法优化。

来源：垃圾邮件特征表据课程讲义；0-1 损失的表述统计学习通行。

为什么 0-1 损失难优化 Why the 0-1 Loss Is Hard

三个原因

1. 梯度为零：阶跃函数几乎处处是常数，$\nabla_w \hat{L} = 0$，梯度下降无从下手；
2. 不可导：在 $w^\top x = 0$ 处跳变；
3. 组合爆炸：在最坏情形下，这个问题是 NP-hard 的。

教科书的原话

"Drawback: difficult to optimize. NP-hard in the worst case." ——这正是课程讲义对这一做法的结论。

所以需要找一个光滑的替代损失（surrogate loss），它把 0-1 损失"包"起来：最小化它就能间接降低误分类率。下一帧给出第一个替代方案。

来源：NP-hard 的结论据课程讲义原文；替代损失的思路统计学习通行。

退一步：把分类当回归 Regression as a Classifier

做法

忽略 $y \in \{0, 1\}$ 的事实，直接用平方损失：

$$\hat{L}(w) = \frac{1}{n} \sum_{i=1}^{n} \left(w^\top x_i - y_i \right)^2,$$

然后令 $\hat{y} = \mathbb{I}[w^\top x > 0.5]$。

好处

有闭式解、好优化，实现只需几行（帧 30 的代码可复用）。

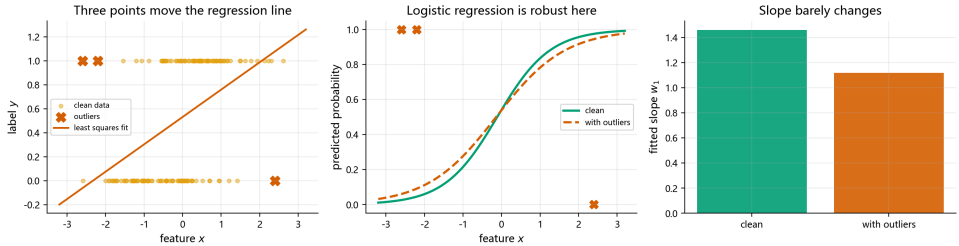
代价

平方损失惩罚"太自信"的预测，因此对离群点 outlier 极其敏感——一个标错且离边界很远的点，会把整条边界拉歪（帧 55 的样本）。

这是一个重要的教训：损失函数的选择会改变模型的鲁棒性，不只是改变"优化好不好做"。

来源：回归当分类的做法及其缺陷据课程讲义与 Bishop 教材。

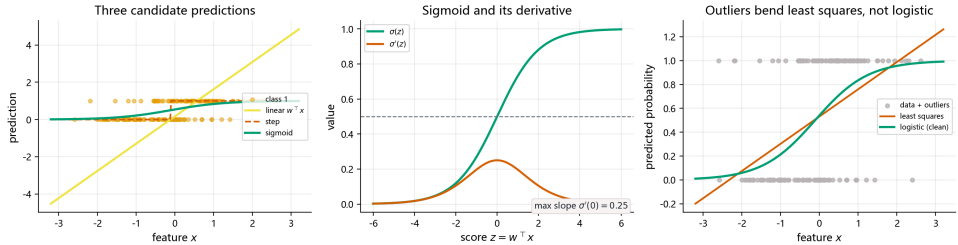
【样本 + 可视化】离群点如何破坏回归分类器 Outliers



左：三个标错的点把最小二乘的直线压弯（阈值从 0 附近移到 -2.3）；中：逻辑回归的边界几乎不动；右：两种方法斜率的直接对比（clean 与 with outliers）。

程序 `16_05_linreg_vs_sigmoid.py` ·样本 `16_two_class.csv`（含三个离群点）

【样本 + 可视化】step、线性与 sigmoid 三条曲线



左：三种候选预测——线性得分无界、step 不连续、sigmoid 有界且光滑；中：sigmoid 与它的导数，拐点处斜率最大（0.25）；右：同一份带离群点的数据，最小二乘被拉弯而逻辑回归基本不动。

程序 `16_05_linreg_vs_sigmoid.py` ·样本 `16_two_class.csv`

sigmoid 的三条性质 Three Properties of the Sigmoid

(1) 有界

$\sigma(a) \in (0,1)$：永不取到 0 或 1，适合当概率。

(2) 对称

$1-\sigma(a) = \sigma(-a)$：“反面”与 “取负数” 等价。

(3) 导数

$\sigma'(a) = \sigma(a)\big(1-\sigma(a)\big)$：梯度只用输出值就能算。

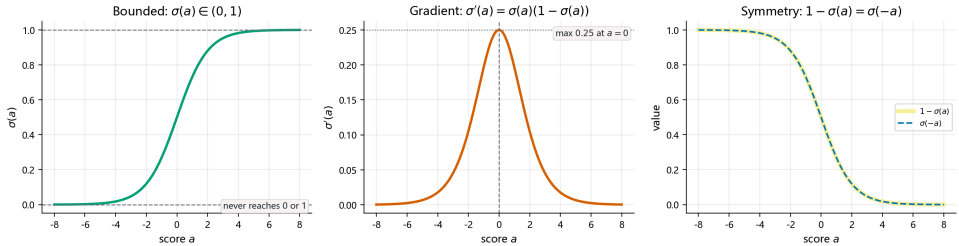
两条公式的推导

(2) $1-\sigma(a) = \frac{e^{-a}}{1+e^{-a}} = \sigma(-a)$；　(3) $\sigma'(a) = \frac{e^{-a}}{(1+e^{-a})^2} = \sigma(a)\big(1-\sigma(a)\big)$.

第 (3) 条是逻辑回归能高效训练的关键：梯度可以只用输出值算出来，不需要再算一遍指数函数 (帧 62 会用到)。

来源：三条性质据课程讲义；推导为本讲展开。

【样本 + 可视化】sigmoid 的形状、导数与对称性



左：值域严格在 $(0, 1)$ 内；中：导数关于 0 对称，最大值 0.25（在 $a = 0$ 处）；右：$1 - \sigma(a)$ 与 $\sigma(-a)$ 完全重合——这就是" 对称性" 的图形表示。

程序 `16_05_linreg_vs_sigmoid.py` ·图由同一份代码生成，便于核对公式

逻辑回归：sigmoid + 极大似然 Logistic Regression

把输出解释为概率

$$P_w(y=1 \mid x)=\sigma(w^\top x), \qquad P_w(y=0 \mid x)=1-\sigma(w^\top x).$$

写出似然并取负对数

利用 $y_i \in \{0,1\}$，可把两种情形合并：

$$\hat{L}(w)=-\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log \sigma(w^\top x_i)+(1-y_i)\log\big(1-\sigma(w^\top x_i)\big)\Big].$$

这正是交叉熵 cross-entropy 损失，也是伯努利 Bernoulli 分布下的 NLL。

术语：逻辑回归 logistic regression = sigmoid 变换 + 交叉熵损失；它做的是分类，名字里的" 回归" 来自它建模的是对数几率（帧 60）。

来源：推导据课程讲义与统计学习教材。

对数几率与系数解释 Log-Odds and Coefficients

对数几率

设 $p = P(y = 1 \mid x)$，则

$$\log \frac{p}{1-p} = w^\top x.$$

左端叫对数几率 log-odds；它关于 x 线性——这就是" 逻辑回归是线性模型" 的准确含义。

系数怎么读

特征 j 增加 1 个单位：

- 对数几率增加 w_j；
- 几率乘以 $\exp(w_j)$，即几率比 odds ratio；
- 本讲样本：$w_1 = 1.17$，几率比 $e^{1.17} = 3.22$ ——几率变为 3.2 倍。

业务建议：汇报逻辑回归时，给几率比而不是给系数；系数是" 对数几率的变化"，非专业人士很难直读。

来源：对数几率与几率比据统计学习教材；数值来自程序 l6_10。

为什么没有闭式解 No Closed-Form Solution

原因

交叉熵损失对 w 是非线性的（sigmoid 是非线性的），令梯度为零会得到一个没有解析解的方程组。

好消息：凸性

逻辑回归的损失是凸函数 convex function：

- 任何局部最优都是全局最优；
- 梯度下降一定收敛（步长合适时）。

所以策略变了：不再"求公式"，而是迭代求解。这是从"线性回归"到"逻辑回归"最重要的一步跨越——也是所有深度学习优化的原型。

来源：凸性与无闭式解据统计学习教材。

交叉熵损失的梯度 The Gradient of the Cross-Entropy

推导（关键一步）

记 $p_i = \sigma(w^\top x_i)$，利用 $\sigma' = \sigma(1-\sigma)$：

$$\frac{\partial}{\partial w}\Big[-\log \sigma(w^\top x_i)\Big] = -\frac{\sigma(1-\sigma)}{\sigma} x_i = -(1-p_i)x_i,$$

$$\frac{\partial}{\partial w}\Big[-\log\big(1-\sigma(w^\top x_i)\big)\Big] = \frac{\sigma(1-\sigma)}{1-\sigma} x_i = p_i x_i.$$

合并两种情形：

$$\nabla_w \hat{L} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)x_i = \frac{1}{n} X^\top\big(\sigma(Xw) - y\big).$$

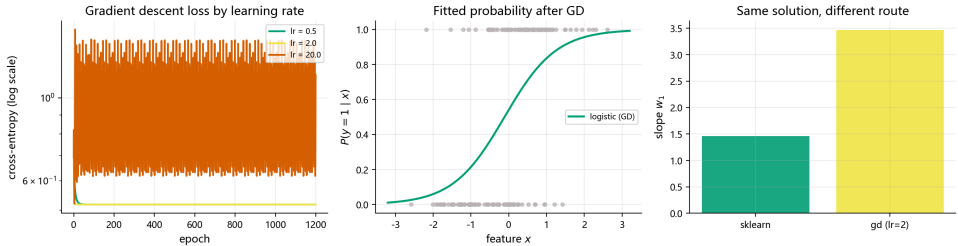
注意这个形式与线性回归一模一样：$X^\top$(预测 − 真值)/n。唯一区别是预测值过了一次 sigmoid——这是"广义线性模型"的共同结构。

【代码】手写梯度下降训练逻辑回归 Gradient Descent by Hand

```python
def logistic_loss(w_vec, Xm, yv, l2=0.0):            # mean cross-entropy plus optional L2
    p = sigmoid(Xm @ w_vec)                          # predicted probabilities
    eps = 1e-12                                      # avoid log(0)
    loss = -np.mean(yv * np.log(p + eps) + (1 - yv) * np.log(1 - p + eps)) # cross-entropy
    loss += 0.5 * l2 * float(w_vec @ w_vec)          # weight decay term
    return float(loss)                               # return the scalar loss
def logistic_grad(w_vec, Xm, yv, l2=0.0):            # analytic gradient
    p = sigmoid(Xm @ w_vec)                          # predicted probabilities
    grad = (Xm.T @ (p - yv)) / len(yv)               # X^T (p - y) / n
    grad += l2 * w_vec                               # gradient of the weight decay
    return grad                                      # return the gradient vector
def train_gd(learning_rate, epochs=1200, l2=0.0):    # full-batch gradient descent
    w_vec = np.zeros(X.shape[1])                     # start from zero weights
    hist = []                                        # loss history per epoch
    for _ in range(epochs):                          # fixed number of full-batch steps
        hist.append(logistic_loss(w_vec, X, two["y"].values, l2)) # record the loss before the update
        w_vec = w_vec - learning_rate * logistic_grad(w_vec, X, two["y"].values, l2) # gradient step
    return np.array(hist), w_vec                     # return history and final weights
```

要点：三行核心——算概率、算损失、算梯度；其余都是记账。完整程序 `16_05_linreg_vs_sigmoid.py`，与 sklearn 结果对比在帧 64。

【样本 + 可视化】收敛曲线与学习率的敏感性 Learning Rate



左：学习率 0.5 与 2.0 都收敛到同一损失（0.518），20.0 振荡发散（损失停在 1.15）；中：梯度下降拟合出的概率曲线；右：与 sklearn 的斜率几乎一致（截距的小差异来自 sklearn 默认的 L2 正则）。

程序 `16_05_linreg_vs_sigmoid.py` ·轨迹表 `16_gd_history.csv`

【代码】PyTorch 自动求导：与手写梯度对账 Autograd

```python
import torch                                          # automatic differentiation
import torch.nn as nn                                 # loss modules
Xt = torch.tensor(X_np, dtype=torch.float64)          # torch version of the design matrix
yt = torch.tensor(y_np, dtype=torch.float64)          # torch version of the labels
w_t = torch.zeros(X_np.shape[1], dtype=torch.float64, requires_grad=True) # parameter with gradients
opt = torch.optim.SGD([w_t], lr=2.0)                  # plain stochastic gradient descent
bce = nn.BCEWithLogitsLoss()                          # numerically stable logistic loss
for _ in range(800):                                  # same number of epochs
    opt.zero_grad()                                   # clear the previous gradients
    logits = Xt @ w_t                                 # linear score
    loss = bce(logits, yt)                            # loss computed in a stable way
    loss.backward()                                   # autograd fills w_t.grad
    opt.step()                                        # apply the update
w_check = torch.tensor([0.1, 0.4], dtype=torch.float64, requires_grad=True) # arbitrary point
loss_check = nn.functional.binary_cross_entropy_with_logits(Xb @ w_check, yb) # loss there
loss_check.backward()                                 # autograd gradient
assert np.allclose(grad_torch, grad_np_b, atol=1e-10), "autograd and analytic gradients disagree"
```

实测：两种路线得到完全相同的系数（0.1627, 1.1709），梯度差 2.8e-17；`BCEWithLogitsLoss` 内部已做 log-sum-exp，所以数值稳定。完整程序 `l6_06_logistic_torch.py`。

优化器家族与早停 Optimisers and Early Stopping

优化器	更新规则（直觉）	适用情形
SGD	沿当前梯度走一步	简单、稳，但慢
Momentum	累积"惯性"，抑制震荡	病态曲面、方向反复
Adam	按参数自适应步长	默认首选，对尺度不敏感

超参数

学习率 learning rate、批量大小 batch size、权重衰减 weight decay——都是超参数 hyper-parameter，需用验证集选。

早停

早停 early stopping：验证损失不再下降就停。它本身是一种隐式正则化（帧 75）。

来源：优化器家族与早停据深度学习实践；术语为本讲统一。

标准化与收敛速度 Standardisation

问题

特征量纲差异大时（年龄以年计、收入以万元计），交叉熵的等高线被拉成细长椭圆，梯度下降会在窄方向上来回震荡。

对策

先做标准化 standardisation：

$$x_j \leftarrow \frac{x_j - \bar{x}_j}{s_j}.$$

好处：(1) 收敛更快；(2) 正则化更公平；(3) 系数可比。

两个坑：(1) 统计量 $\bar{x}, s$ 只能来自训练集，否则泄漏；(2) 截距不标准化，正则化时也通常不惩罚截距。

来源：标准化的作用与两个坑据机器学习实践；与 L5 的泄漏清单一致。

多分类：softmax 回归 Softmax Regression

从 sigmoid 到 softmax

对 K 类，每类有自己的权重 w_k，输出

$$P(y=k \mid x)=\frac{e^{w_k^\top x}}{\sum_{j=1}^{K} e^{w_j^\top x}}.$$

分母保证 $\sum_k P=1$；$K=2$ 时退化为 sigmoid。

损失与梯度

多类交叉熵：

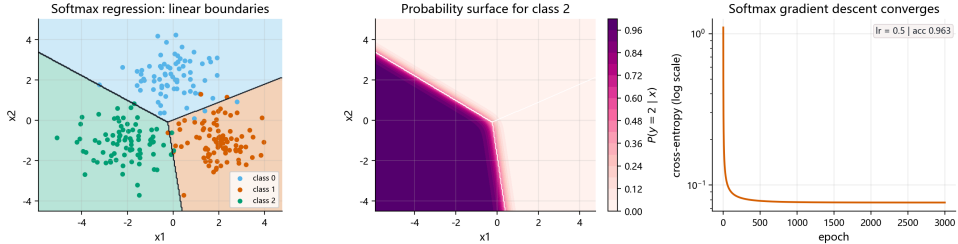
$$\hat{L}=-\frac{1}{n}\sum_i \log P(y_i \mid x_i),$$

梯度形式仍然是 $X^\top(P-Y)/n$（P 为概率矩阵，Y 为 one-hot）。

工程注意：实现时用 log-sum-exp（帧 48）算分母，否则大 logits 会溢出；参数化时可省掉一个冗余参数。

来源：softmax 回归据 Bishop 与统计学习教材；数值注意为本讲补充。

【样本 + 可视化】softmax 的决策区域与概率面



三类二维样本（$n = 240$）：左：线性决策边界把平面分成三块；中：类别 2 的概率面，边界处约 0.5；右：梯度下降的损失曲线。**sklearn 与手写实现准确率 0.963 vs 0.967**，差别只来自正则与收敛精度。

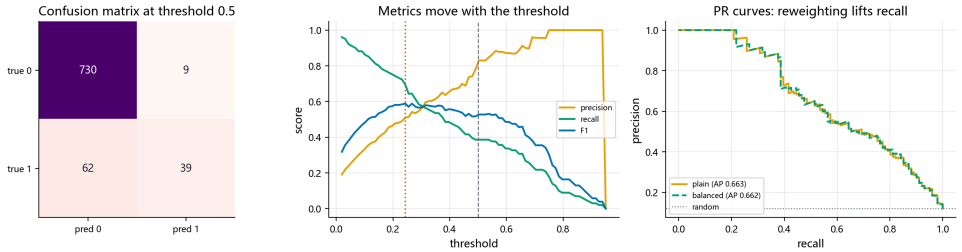
程序 `16_07_softmax_regression.py` ·样本 `16_three_class.csv` ·轨迹 `16_softmax_history.csv`

【代码】softmax 的手写实现与 sklearn 对照

```python
def softmax_matrix(Z):                          # row-wise stable softmax
    Z = np.asarray(Z, dtype=float)              # work with a float array
    Z = Z - Z.max(axis=1, keepdims=True)        # remove the overflow risk
    E = np.exp(Z)                               # safe exponentials
    return E / E.sum(axis=1, keepdims=True)     # normalise every row
def softmax_loss_grad(W, Xm, Y_oh):             # cross-entropy and gradient
    P = softmax_matrix(Xm @ W)                  # class probabilities for every row
    eps = 1e-12                                 # avoid log(0)
    loss = -np.mean(np.sum(Y_oh * np.log(P + eps), axis=1)) # mean cross-entropy over the rows
    grad = Xm.T @ (P - Y_oh) / len(Xm)          # gradient with respect to W
    return float(loss), grad                    # return both
W = np.zeros((Xa.shape[1], 3))                  # weight matrix (parameters per class)
for _ in range(epochs):                         # full-batch gradient descent
    loss, grad = softmax_loss_grad(W, Xa, Y_oh) # evaluate loss and gradient
    W = W - lr * grad                           # gradient step
sk = LogisticRegression(multi_class="multinomial", max_iter=2000) # multinomial = softmax regression
```

要点：梯度与二分类同构——$X^\top(P-Y)/n$，只是 P, Y 变成矩阵。`multi_class="multinomial"` 才是真正的 softmax；默认的 `ovr` 是"一对多"。完整程序 `16_07_softmax_regression.py`。

【样本 + 可视化】类别不平衡与重加权 Class Imbalance



一个 8% 正例的样本 ($n = 2400$)：左：阈值 0.5 时的混淆矩阵——大量漏判藏在右下角；中：阈值一动，精确率/召回率/F1 全变（最优 F1 在 0.27 附近）；右：`class_weight="balanced"` 明显抬高 PR 曲线。

程序 `l6_09_metrics_roc.py` ·样本 `l6_imbalanced.csv` ·阈值表 `l6_threshold_metrics.csv`

阈值选择与代价敏感 Thresholds and Costs

阈值不是 0.5

0.5 只是"概率到标签"的默认切点。业务上应根据两类错误的代价来选：

$$t^* = \arg\min_t \big[\text{成本}(t)\big],$$

成本 $= \text{FN} \times C_{FN} + \text{FP} \times C_{FP}$。

两类错误

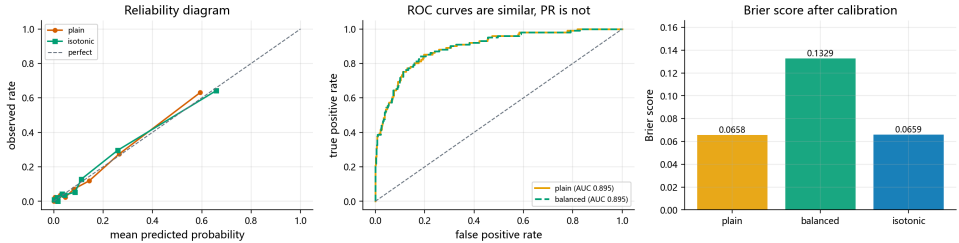
- 第一类错误 Type I error（误报 FP）：把好事判成坏事；
- 第二类错误 Type II error（漏判 FN）：把坏事放过。

降低一类通常抬高另一类——所以必须按业务定价。

项目里的写法：先给出 C_{FN} 与 C_{FP} 的依据（监管罚金、客诉成本、人工复核成本），再报告最优阈值与该阈值下的混淆矩阵。

来源：阈值与代价敏感据业务建模实践；衔接 L5 的评估协议。

【样本 + 可视化】概率校准与 ROC/PR Calibration



左：可靠性曲线——未校准的模型在低概率区系统性偏高，等渗回归 isotonic regression 后贴近对角线；中：ROC 曲线两条几乎重合（AUC 0.870 vs 0.871），但 PR 曲线差别明显；右：布里尔分数 Brier score 从 0.0659 降到 0.0649。

程序 `16_09_metrics_roc.py` ·汇总表 `16_metrics_summary.csv`

小结：线性回归与逻辑回归对照 Two Linear Models

维度	线性回归 Linear regression	逻辑回归 Logistic regression
任务	回归（连续 y）	二分类（$y \in \{0, 1\}$）
输出	$w^\top x$	$\sigma(w^\top x)$，概率
损失	平方损失（高斯 MLE）	交叉熵（伯努利 MLE）
解	闭式解 $w = (X^\top X)^{-1} X^\top y$	无闭式解，梯度下降
梯度	$\frac{2}{n} X^\top (Xw - y)$	$\frac{1}{n} X^\top (\sigma(Xw) - y)$
系数解释	一单位变化引起的 y 变化	一单位变化引起的对数几率变化

同一套框架（假设类 + 损失 + 优化），换一个" 输出变换 + 损失" 就得到新模型。下一部分给这个框架加上复杂度控制——正则化。

来源：对照表为本讲整理；术语与公式均在前文推导过。

为什么需要正则化 Why Regularise

三个症状

- 特征数 P 接近或超过样本数 N：$X^\top X$ 不可逆；
- 系数很大、符号反复：对数据极敏感；
- 训练误差很低，验证误差很高：过拟合 overfitting。

思路

给模型加一个复杂度预算 complexity budget：不允许系数自由变大。预算越紧，模型越"简单"，方差越小、偏差越大（帧 83）。

正则化不是"补救措施"，而是建模的一部分：它把你的先验（"系数不该太大"）写成数学。这也是它和特征选择、早停能互相替代的原因。

来源：正则化的动机据统计学习教材；复杂度预算的比喻为本讲补充。

惩罚回归：目标函数 Penalised Regression

形式

在损失后加一个惩罚项 penalty：

$$\min_{w} \underbrace{\frac{1}{n}\sum_{i}(y_i - w^\top x_i)^2}_{\text{残差平方和}} + \underbrace{\lambda\,\Omega(w)}_{\text{惩罚项}}.$$

$\lambda \geq 0$ 是超参数 hyper-parameter，控制" 拟合" 与" 简单" 的权衡。

三种常见的 Ω

- $\|w\|_1 = \sum_j |w_j|$：LASSO （帧 77）；
- $\|w\|_2^2 = \sum_j w_j^2$：Ridge （帧 78）；
- 两者混合：Elastic Net （帧 79）。

注意：惩罚项通常不含截距；$\lambda = 0$ 时退回普通最小二乘。

关键术语

惩罚回归 penalised regression、收缩 shrinkage、稀疏 sparsity。

来源：惩罚回归的表述（统计学习教材 ESL 第 3 章）。

LASSO：L1 惩罚与稀疏解 LASSO

形式

$$\min_{w} \frac{1}{n}\sum_{i}(y_i - w^\top x_i)^2 + \lambda \sum_{j}|w_j|.$$

三条性质

- λ 增大时，部分系数恰好变成 0；
- 因此自动完成特征选择 feature selection；
- 解是稀疏的 sparse solution——只留少数变量。

直觉（几何）

L1 的等高线是菱形，相切点常在顶点上——那里某些坐标恰为 0。

原书的比喻

"Capitalist"——in doubt, give all to one. （面对相关特征，L1 倾向于只留一个。）

业务价值：LASSO 给出的变量清单本身就是结论——" 这 3 个因素比那 20 个更重要" 是报告里常见的一句话。

来源：LASSO 由 Tibshirani (1996) 提出；几何与比喻据 ESL 与课程讲义。

Ridge：L2 惩罚与收缩 Ridge Regression

形式

$$\min_{w} \frac{1}{n}\sum_{i}(y_i - w^\top x_i)^2 + \lambda \sum_{j} w_j^2.$$

它有闭式解：$w = (X^\top X + \lambda I)^{-1} X^\top y$——$\lambda I$ 让矩阵可逆，绕开了共线性问题。

性质

- 系数向零收缩 shrinkage，但不会恰好为零；
- 对共线特征" 平均分配" 权重——"Socialist"（in doubt, distribute to multiple）；
- 可解释为高斯先验下的贝叶斯后验。

实测对照（本讲前列腺样本 + 6 个噪声列，14 个特征）：Ridge 保留 14 个、Lasso 保留 13 个；测试 RMSE：Ridge 0.724、Lasso 0.705、Elastic Net 0.712。

来源：Ridge 的闭式解与贝叶斯解释统计学习教材；实测数据来自程序 l6_08。

Elastic Net：两者的混合 Elastic Net

形式

$$\min_{w} \frac{1}{n}\sum_{i}(y_i - w^\top x_i)^2 + \lambda\Big(\alpha\sum_{j}|w_j| + (1-\alpha)\sum_{j} w_j^2\Big).$$

$\alpha \in [0,1]$ 控制 L1 与 L2 的比例。

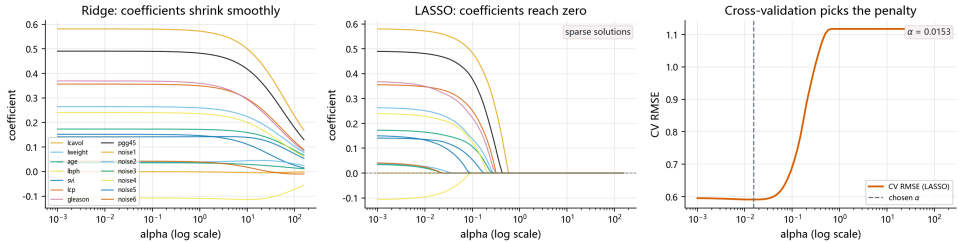
什么时候用

- 特征成组高度相关：L1 会随机留一个，Elastic Net 会成组保留；
- 特征数远大于样本数（$P \gg N$）；
- 既想要稀疏、又想要稳定。

超参数变成两个（λ 与 α），所以需要的验证数据更多。实践建议：先固定 $\alpha = 0.5$ 调 λ，再网格搜两个。

来源：Elastic Net 由 Zou & Hastie (2005) 提出；实践建议为本讲补充。

【样本 + 可视化】正则化路径与交叉验证选 λ



左：Ridge 的系数平滑收缩（8 个真实特征 + 6 个噪声列）；中：LASSO 的系数逐个归零，噪声列最先消失；右：交叉验证误差随 λ 变化，虚线是选中的 λ（本样本 0.0178）。

程序 `l6_08_regularization_cv.py` ·样本 `l6_prostate.csv` ·对照表 `l6_regularization_scores.csv`

【代码】LASSO / Ridge / Elastic Net 的交叉验证

```python
from sklearn.linear_model import (RidgeCV, LassoCV, ElasticNetCV, # CV-fitted penalties
                                  Ridge, Lasso)               # single-strength penalties for the paths
alphas = np.logspace(-3, 2.2, 80)                             # penalty grid
cv = KFold(n_splits=8, shuffle=True, random_state=42) # eight folds for a stable CV curve
models = {"Ridge": RidgeCV(alphas=alphas, cv=cv),     # L2 penalty
          "Lasso": LassoCV(alphas=alphas, cv=cv, max_iter=50000, random_state=42), # L1 penalty
          "ElasticNet": ElasticNetCV(alphas=alphas, l1_ratio=[0.2, 0.5, 0.8], cv=cv, # mixed penalty
                                     max_iter=50000, random_state=42)} # generous solver budget
for name, model in models.items():                            # loop over the three penalties
    pipe = Pipeline([("scale", StandardScaler()), ("model", model)]) # scaling inside the pipeline
    pipe.fit(X_train, y_train)                                # fit on the training split
    pred = pipe.predict(X_test)                               # predictions on the hold-out split
    mdl = pipe.named_steps["model"]                           # the fitted penalty object
    coefs = mdl.coef_ if mdl.coef_.ndim == 1 else mdl.coef_[0] # coefficients (flatten for ElasticNet)
    nonzero = int(np.sum(np.abs(coefs) > 1e-8))               # count surviving features
```

要点：标准化必须在 Pipeline 里，否则测试集的统计量会泄漏到训练；交叉验证在训练集内部完成，测试集只在最后用一次。完整程序 `l6_08_regularization_cv.py`。

如何选 λ：交叉验证与 1-SE 规则 Choosing the Strength

交叉验证

在训练集内部切 K 折，轮流留一折验证，取平均误差最小的 λ。**测试集不能参与选 λ**——否则样本外误差会被乐观高估。

1-SE 规则

选误差在一个标准误之内的最简单（λ 最大）的模型：宁可简单一点，换取稳定性。

做法	后果
在测试集上选 λ	乐观偏差，上线后骤降（L5 帧 42 的清单）
用交叉验证 + 1-SE	稳健，报告里可解释
固定 λ 不解释	审稿人会问" 为什么是这个数"

来源：交叉验证与 1-SE 规则据 ESL；偏差说明衔接 L5。

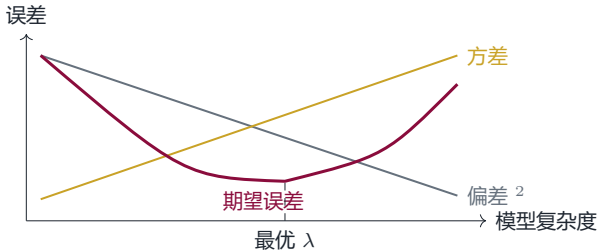
正则化与偏差-方差 Regularisation as Bias for Variance

一句话

正则化主动引入偏差，换取方差的下降。只要方差下降得多，总误差就下降。

什么时候不划算

当真实关系很强、样本很足时，惩罚会压掉真实信号，造成欠拟合——这就是 λ 不能太大的原因。



来源：偏差-方差权衡据 ESL；本图为本讲自绘。

偏差-方差分解 The Bias-Variance Decomposition

分解式

对平方损失，期望预测误差可写成三项：

$$\mathbb{E}\big[(y-\hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}\hat{f}(x)-f(x)\big)^2}_{\text{偏差}^2\ \mathrm{Bias}^2} + \underbrace{\mathbb{E}\big[\hat{f}(x)-\mathbb{E}\hat{f}(x)\big]^2}_{\text{方差 Var}} + \underbrace{\sigma^2}_{\text{噪声}}.$$

读法

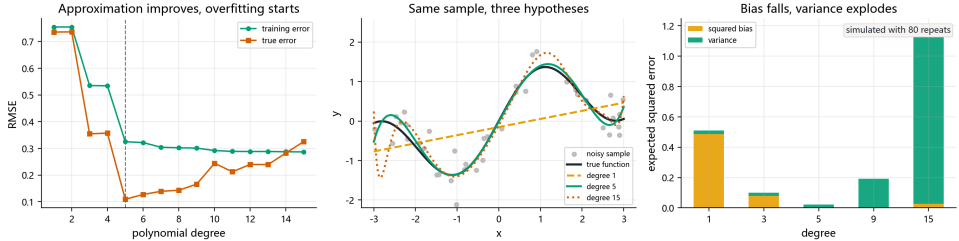
- 噪声 σ^2：不可约，任何模型都消不掉；
- 偏差：模型太简单造成的系统偏离；
- 方差：换一份样本就变很多。

与正则化的关系

正则化压方差、抬偏差；数据集变大主要压方差；更复杂的假设类压偏差、抬方差。

来源：分解式据 ESL 第 7 章；读法为本讲整理。

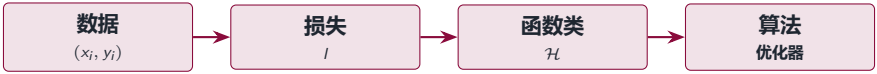
【样本 + 可视化】近似—过拟合权衡 Approximate or Overfit



左：多项式次数上升，训练误差单调下降，真实误差先降后升（最优在 5 次）；中：同一份样本的三种拟合（1 次欠拟合、5 次合适、15 次过拟合）；右：用 80 次重复模拟得到的偏差 ^{2} 与方差分解——9 次时方差从 0.019 跳到 0.822。

程序 `l6_11_overfit_curves.py` ·样本 `l6_poly_sample.csv` ·分解表 `l6_bias_variance.csv`

预测问题的统一框架 The Prediction Problem



四要素

四样都定了，模型就定了：**数据、损失、函数类、算法。**

术语

经验风险最小化 ERM；正则化 regularisation 是对函数类的额外约束。

为什么有用

任何"新模型"都可以用这四个格子定位——它改的是哪一格？

来源：四要素框架据 ESL 与课程讲义；"改哪一格"的提法为本讲补充。

【术语】计量经济学的说法 vs 机器学习的说法

计量经济学 Econometrics	机器学习 Machine Learning	说明
因变量 dependent variable	标签 label / 目标 target	要预测的量 y
自变量 independent variable	特征 feature / 协变量 covariate	输入 x
估计 estimate	训练 train / 拟合 fit	从数据求参数
预测 predict	推断 infer（预测时）	对新样本输出
标准误 standard error	验证误差 validation error	不确定性的度量
系数 coefficient	权重 weight	参数 w_j
拟合优度 goodness of fit	样本外 R^2 out-of-sample R^2	拟合 vs 泛化

同一件事两种语言。写项目报告时选定一套并全篇统一，并在附录里给出这张对照表——评审老师可能来自任一传统。

来源：对照表据 Mullainathan & Spiess (2017, JEP) 与 Athey (2017)；说明为本讲补充。

回归的平方误差损失 Squared-Error Loss in Practice

两个经典应用

- 房价预测：$y = \log(\text{房价})$，特征为面积、房龄、地段；
- 消费预测：$y = \log(\text{消费})$，特征为收入、家庭规模。

两者都用 l_2 损失，并报告RMSE 与 R^2。

为什么常取对数

- 右偏分布被拉回近似对称；
- 系数可解释为百分比变化（弹性）；
- 误差更接近同方差。

注意：取对数后，$\hat{y}$ 在对数尺度上无偏，但在原尺度上是有偏的（Jensen 不等式）——报告里要么说清尺度，要么做校正。

来源：案例据 Mullainathan & Spiess (2017, JEP)；对数尺度的偏差为本讲补充。

【代码】混淆矩阵、阈值扫描与 ROC/PR

```python
from sklearn.metrics import (confusion_matrix, roc_curve, auc, # evaluation metrics
                        precision_recall_curve, average_precision_score,
                        precision_score, recall_score, f1_score,
                        brier_score_loss, accuracy_score)
thresholds = np.linspace(0.02, 0.95, 80)              # threshold grid
for t in thresholds:                                  # sweep the grid
    pred = (proba >= t).astype(int)                   # hard labels
    rows.append({"threshold": round(float(t), 3),     # threshold value
                 "precision": round(float(precision_score(y_test, pred, zero_division=0)), 4), # precision
                 "recall": round(float(recall_score(y_test, pred, zero_division=0)), 4), # recall
                 "f1": round(float(f1_score(y_test, pred, zero_division=0)), 4)}) # F1
weighted = Pipeline([("scale", StandardScaler()),     # same preprocessing
                     ("model", LogisticRegression(max_iter=2000, class_weight="balanced"))]) # reweighted loss
prec_w, rec_w, _ = precision_recall_curve(y_test, proba_w) # PR curve for the reweighted model
fpr_u, tpr_u, _ = roc_curve(y_test, proba)            # ROC curve for the plain model
calibrated = CalibratedClassifierCV(                  # isotonic calibration with internal CV
    Pipeline([("scale", StandardScaler()), ("model", LogisticRegression(max_iter=2000))]), # base model
    method="isotonic", cv=5)                          # isotonic regression
```

实测：AUC 0.870、AP 0.601、Brier 0.0659；重加权后 AP 略降但召回显著提高，等渗校准把 Brier 降到 0.0649。完整程序 `16_09_metrics_roc.py`。

监督学习者的结构 Structure of a Supervised Learner



函数类

线性、多项式、树、神经网络——决定能表达什么。

正则项

L1/L2、早停、dropout——决定有多保守。

优化算法

闭式解、梯度下降、坐标下降——决定多快找到解。

这张图是本讲的收束：任何监督学习算法都可以填进这三格。L7 的树与集成、L9 的神经网络，都是在这三格上做文章。

来源：三要素结构据课程讲义（第 80 页）；解读为本讲补充。

线性模型家族全景 The Linear-Model Family

模型	输出变换	损失	正则化版本
线性回归 OLS	恒等	平方	Ridge / Lasso / Elastic Net
逻辑回归	sigmoid	交叉熵	带 L2 惩罚的 `LogisticRegression`
softmax 回归	softmax	多类交叉熵	同上（多类）
泊松回归	exp	泊松 NLL	带惩罚的 GLM

共同点：都是广义线性模型 generalised linear model——线性得分 + 一个" 连接函数" + 对应分布的 NLL。掌握一个，其余只是换变换与换损失。

来源：GLM 与线性模型家族据统计学习教材；对照表为本讲整理。

小结：本讲一张图 One Picture of Lecture 6



三个”为什么”

- 损失为什么是平方/交叉熵？→ 似然（帧 43、59）
- 系数怎么解释？→ 对数几率与几率比（帧 60）
- 复杂度怎么控？→ 正则化与交叉验证（帧 76–82）

下一部分

工具与术语：什么时候用推断视角 (statsmodels)、什么时候用预测视角（sklearn），以及一张79 条的术语表。

来源：本图为本讲内容的压缩；各帧均在前文展开。

术语表 A–D Glossary A–D

中文	English
0-1 损失	0-1 loss
近似误差	approximation error
AUC	area under the ROC curve
增广技巧	augmented trick
自动微分	automatic differentiation
布里尔分数	Brier score
批量	batch
偏差-方差分解	bias-variance decomposition
二分类	binary classification

中文	English
类别不平衡	class imbalance
条件数	condition number
条件似然	conditional likelihood
置信区间	confidence interval
混淆矩阵	confusion matrix
凸函数	convex function
交叉熵	cross-entropy
交叉验证	cross-validation

术语共 17 条（分组：A–D）；完整表含符号、公式与一句话解释，见 `l6_glossary.csv` 与站点讲义页。

术语表 E–M Glossary E–M

中文	English
弹性网	elastic net
经验损失	empirical loss / empirical risk
期望损失	expected loss / risk
特征向量	feature vector
泛化差距	generalization gap
生成式模型	generative model
梯度下降	gradient descent
超参数	hyper-parameter
超平面	hyperplane
假设类	hypothesis class
独立同分布	independent and identically distributed (i.i.d.)
截距	intercept / bias
等渗回归	isotonic regression
KL 散度	Kullback-Leibler divergence
L1 惩罚	L1 penalty / LASSO
L2 惩罚	L2 penalty / Ridge

中文	English
标签	label
学习率	learning rate
似然	likelihood
线性分类器	linear classifier
线性回归	linear regression
对数似然	log-likelihood
对数几率	log-odds
对数求和指数	log-sum-exp
逻辑回归	logistic regression
机器学习	machine learning
间隔	margin
极大似然估计	maximum likelihood estimation (MLE)
均方误差	mean squared error (MSE)
多分类	multi-class classification
多重共线性	multicollinearity

术语共 31 条（分组：E–M）。完整表含符号、公式与一句话解释，见 `16_glossary.csv` 与站点讲义页。

术语表 N–Z Glossary N–Z

中文	English
负对数似然	negative log-likelihood (NLL)
正规方程	normal equation
几率比	odds ratio
过拟合	overfitting
Platt 缩放	Platt scaling
p 值	p-value
惩罚回归	penalized regression
精确率	precision
概率校准	probability calibration
投影	projection
ROC 曲线	ROC curve
召回率	recall / sensitivity
正则化	regularization
表征学习	representation learning
收缩	shrinkage
sigmoid 函数	sigmoid function
softmax	softmax
稀疏解	sparse solution
平方损失	squared loss / l_2 loss
标准误	standard error
阶跃函数	step function
监督学习	supervised learning
第一类错误	Type I error (false positive)
第二类错误	Type II error (false negative)
测试数据	test data
阈值	threshold
训练数据	training data
下溢	underflow
无监督学习	unsupervised learning

术语共 29 条（分组：N–Z）。完整表含符号、公式与一句话解释，见 16_glossary.csv 与站点讲义页。

工具栈与各自的分工 The Tool Stack

工具	擅长	本讲用在哪
NumPy	手写公式、验证推导	正规方程、梯度下降、softmax
scikit-learn	预测、流水线、调参	交叉验证、正则化、指标
statsmodels	推断：标准误、p 值、区间	帧 94 的系数表
PyTorch	自动微分、GPU、深度网络	帧 65 的 autograd 对照

选型口诀：要" 解释" 用 statsmodels，要" 预测" 用 sklearn，要" 验证自己的推导" 用 NumPy，要" 上规模" 用 PyTorch。

来源：工具分工据各库官方文档与本讲实践。

【代码】statsmodels：推断视角 Inference View

```python
import statsmodels.api as sm                          # OLS and Logit with full inference output
X = add_intercept(prostate[cols].values)              # augmented design matrix
y = prostate["lpsa"].values                           # target
ols = sm.OLS(y, X).fit()                              # fitted model with full statistics
ci = ols.conf_int()                                   # 95 per cent confidence intervals
table = pd.DataFrame({"term": names,                  # term name
                      "coef": np.round(ols.params, 4),   # estimated coefficient
                      "std_err": np.round(ols.bse, 4),   # standard error
                      "t_value": np.round(ols.tvalues, 3), # t statistic
                      "p_value": np.round(ols.pvalues, 4), # two-sided p value
                      "ci_low": np.round(ci[:, 0], 4),   # lower bound of the interval
                      "ci_high": np.round(ci[:, 1], 4)}) # upper bound of the interval
logit = sm.Logit(yl, Xl).fit(disp=False)              # maximum likelihood logistic regression
logit_table = pd.DataFrame({"term": ["intercept", "x"], # term names
                        "coef": np.round(logit.params, 4), # log-odds coefficients
                        "odds_ratio": np.round(np.exp(logit.params), 4), # exponentiated coefficients
                        "p_value": np.round(logit.pvalues, 4)}) # p values
print("pseudo R^2 %.4f | log-likelihood %.4f" % (logit.prsquared, logit.llf)) # report the fit
```

实测：OLS 的 R^2 0.767（调整后 0.746，F=36.3）；Logit 的系数 1.171、几率比 3.225、p 值 $<$ 0.0001、伪 R^2 0.181。完整程序 `16_10_statsmodels_inference.py`。

本讲的十个常见错误 Ten Common Mistakes

1. 忘记截距，或把截距一起标准化；
2. 用 $X^\top X$ 求逆而不是 `solve`/`lstsq`；
3. 不标准化就做正则化（惩罚不公平）；
4. 在测试集上选 λ 或调阈值；
5. 把系数当因果效应（帧 60 的对数几率不是因果）；
6. 只看准确率，不看混淆矩阵与 PR；
7. 报告概率却不做校准（帧 73）；
8. 用 0.5 当阈值而不问两类错误的代价；
9. 拿训练误差证明模型好（帧 19、85）；
10. 把"逻辑回归"当成回归预测连续值。

自检：交报告前把这十条过一遍，能省掉大部分"老师一眼看出"的问题。

来源：错误清单为本讲整理，多数来自常见审稿意见。

与后续讲次的衔接 Links to Later Lectures

讲次	与本讲的关系
L7 监督学习算法	支持向量机 = 合页损失 + 间隔；树 = 分段常数函数类
L8 无监督与文本	聚类与降维里的距离、正则化思想
L9 深度学习	多层的线性 +sigmoid 堆叠；autograd 与优化器
L12 因果推断	线性模型当" 调整" 工具；系数何时可作因果解读
L10/L11 营销与 NLP	逻辑回归是 CTR 预估与文本分类的基线

一句话：本讲给的是" 骨架"，后面每一讲都在骨架上加血肉——但换的往往只是" 函数类" 那一格(帧 90)。

来源：衔接关系据课程大纲的章节安排。

结语：三句话讲清一个线性模型 Wrap-up

三句话

1. **假设类**：模型在哪些函数里找？（线性、线性 + sigmoid、线性 + softmax）
2. **损失**：为什么是这个损失？（因为它是一个概率假设的负对数似然）
3. **求解与校验**：怎么解？（闭式解或梯度下降）怎么确认没错？（与参考实现对账、看校准与样本外误差）

自检清单

能推出正规方程？能说出 $\sigma' = \sigma(1-\sigma)$ 的用途？能解释几率比 3.22？

下讲预告

L7：支持向量机、树与集成——换的是"函数类"那一格。

谢谢！问题与项目讨论：`Xieer.dai@szu.edu.cn`；本讲全部代码、样本与术语表见课程仓库 `website/code/lecture06/`。

来源：本讲总结为本讲整理；下讲预告据课程大纲。